

```

        if self.moving_left:
            self.x -= self.settings.ship_speed

        # Update rect object from self.x.
        ❷ self.rect.x = self.x

def blitme(self):
    --snip--

```

We create a settings attribute for Ship, so we can use it in `update()` ❶. Because we're adjusting the position of the ship by fractions of a pixel, we need to assign the position to a variable that can have a float assigned to it. You can use a float to set an attribute of a rect, but the rect will only keep the integer portion of that value. To keep track of the ship's position accurately, we define a new `self.x` ❷. We use the `float()` function to convert the value of `self.rect.x` to a float and assign this value to `self.x`.

Now when we change the ship's position in `update()`, the value of `self.x` is adjusted by the amount stored in `settings.ship_speed` ❸. After `self.x` has been updated, we use the new value to update `self.rect.x`, which controls the position of the ship ❹. Only the integer portion of `self.x` will be assigned to `self.rect.x`, but that's fine for displaying the ship.

Now we can change the value of `ship_speed`, and any value greater than 1 will make the ship move faster. This will help make the ship respond quickly enough to shoot down aliens, and it will let us change the tempo of the game as the player progresses in gameplay.

Limiting the Ship's Range

At this point, the ship will disappear off either edge of the screen if you hold down an arrow key long enough. Let's correct this so the ship stops moving when it reaches the screen's edge. We do this by modifying the `update()` method in Ship:

```

ship.py
def update(self):
    """Update the ship's position based on movement flags."""
    # Update the ship's x value, not the rect.
    ❶ if self.moving_right and self.rect.right < self.screen_rect.right:
        self.x += self.settings.ship_speed
    ❷ if self.moving_left and self.rect.left > 0:
        self.x -= self.settings.ship_speed

    # Update rect object from self.x.
    self.rect.x = self.x

```

This code checks the position of the ship before changing the value of `self.x`. The code `self.rect.right` returns the *x*-coordinate of the right edge of the ship's rect. If this value is less than the value returned by `self.screen_rect.right`, the ship hasn't reached the right edge of the screen ❶. The same goes for the left edge: if the value of the left side of the rect is greater than 0, the ship hasn't reached the left edge of the screen ❷. This ensures the ship is within these bounds before adjusting the value of `self.x`.